

Yesterday 3:18 PM

Why use ExecutorService instead of creating threads manually?

execute() vs submit().

Runnable vs Callable.

Future vs CompletableFuture.

What is a thread pool?

Why are thread pools useful?

FixedThreadPool vs CachedThreadPool.

ScheduledExecutorService.

What happens when the thread pool queue is full?

What is thread pool starvation?

Show more ▾



Absolutely. For a 5-year Java backend interview, don't just give definitions—explain **why**, **trade-offs**, **production implications**, and **examples**.

1. Why use ExecutorService instead of creating threads manually?

Instead of doing this for every task:

 Java



```
new Thread(() -> {  
    // task  
}).start();
```

we use:

 Java



```
ExecutorService executor = Executors.newFixedThreadPool(10);  
  
executor.submit(() -> {
```

```
// task  
});
```

Why?

1. Thread reuse

- Creating a thread is relatively expensive.
- Thread pools reuse existing threads.

2. Control over concurrency

- You can limit how many tasks execute simultaneously.

3. Task management

- `ExecutorService` manages task submission, queuing and execution.

4. Return results

- `submit()` gives a `Future`.

5. Graceful shutdown

- You can shut down the executor cleanly.

Interview answer

"I prefer `ExecutorService` because it separates task submission from thread management. Instead of creating an unlimited number of threads, I can control concurrency through a bounded thread pool, reuse threads, handle results through `Future`, and manage shutdown gracefully."

2. `execute()` vs `submit()`

Both submit tasks to an executor.

`execute()`

 `</> Java`

```
executor.execute(() -> {  
    System.out.println("Processing");  
});
```

Returns:

void



Primarily used for `Runnable` .

`submit()`

</> **Java**



```
Future<?> future = executor.submit(() -> {  
    System.out.println("Processing");  
});
```

Returns:

Future



It can accept:

- `Runnable`
- `Callable`

Example:

</> **Java**



```
Future<Integer> future =  
    executor.submit(() -> 10 + 20);  
  
Integer result = future.get();
```

Important difference: exception handling

With `execute()` :

</> **Java**



```
executor.execute(() -> {  
    throw new RuntimeException("Error");  
});
```

The exception is handled by the thread's uncaught-exception mechanism.

With `submit()` :

</> Java



```
Future<?> future = executor.submit(() -> {  
    throw new RuntimeException("Error");  
});
```

The exception is captured inside the `Future` .

You see it when:

</> Java



```
future.get();
```

which throws `ExecutionException` .

Interview answer

" `execute()` is fire-and-forget and returns nothing. `submit()` returns a `Future`, so I can track completion, retrieve results, cancel the task, and observe exceptions."

3. Runnable vs Callable

Runnable

</> Java



```
Runnable task = () -> {  
    System.out.println("Processing");  
};
```

Characteristics:

- No return value
- Cannot directly throw checked exceptions
- Method:

</> Java



```
void run()
```

Callable

</> Java



```
Callable<Integer> task = () -> {  
    return 100;  
};
```

Characteristics:

- Returns a value
- Can throw checked exceptions
- Method:

</> Java



```
V call() throws Exception
```

Example:

</> Java



```
Future<Integer> future = executor.submit(task);  
  
Integer result = future.get();
```

Interview answer

"Runnable is appropriate for tasks where I don't need a result, while Callable is useful when the task produces a result or needs to throw checked exceptions."

4. Future VS CompletableFuture

This is a very important 5-year interview question.

Future

</> Java



```
Future<String> future =  
    executor.submit(() -> callService());
```

```
String result = future.get();
```

Problem:

</> Java



```
future.get();
```

is blocking.

You cannot easily express:

Call A

↓

Call B

↓

combine result



with Future .

CompletableFuture

</> Java



```
CompletableFuture  
    .supplyAsync(() -> callService())  
    .thenApply(result -> transform(result))  
    .thenAccept(result -> save(result));
```

It supports:

- asynchronous composition
- chaining
- callbacks
- combining multiple tasks
- exception handling

Example:

</> Java



```

CompletableFuture<String> a =
    CompletableFuture.supplyAsync(() -> serviceA());

CompletableFuture<String> b =
    CompletableFuture.supplyAsync(() -> serviceB());

CompletableFuture<String> result =
    a.thenCombine(b, (x, y) -> x + y);

```

Interview answer

"Future is useful for obtaining the result of an asynchronous task, but its API is relatively blocking and difficult to compose. CompletableFuture provides non-blocking callbacks, chaining, combining multiple asynchronous operations and better exception handling."

5. What is a thread pool?

A thread pool is a group of reusable worker threads managed by an executor.

Instead of:

```

Request → create thread
Request → create thread
Request → create thread

```



we have:

```

Tasks - Queue  ┌─ Worker 1
                └─ Worker 2
                └─ Worker 3
                └─ Worker 4

```



Tasks are submitted to the executor.

Workers take tasks from the queue and execute them.

6. Why are thread pools useful?

Main benefits:

1. Thread reuse

Avoid repeated thread creation/destruction.

2. Limit concurrency

Suppose you have:

</> Java



```
Executors.newFixedThreadPool(20);
```

At most approximately 20 tasks execute concurrently.

3. Protect the system

Without a limit:

10 requests

100 requests

10,000 requests

100,000 requests

↓

too many threads

↓

CPU/memory exhaustion



4. Better resource management

You can control:

- pool size
- queue size
- rejection policy
- thread lifetime

7. FixedThreadPool vs CachedThreadPool

FixedThreadPool

</> Java



```
ExecutorService executor =
```

```
Executors.newFixedThreadPool(10);
```

Creates a pool with a fixed number of worker threads.

10 workers



Task Queue



Good when you want **controlled concurrency**.

CachedThreadPool

</> Java

```
ExecutorService executor =  
    Executors.newCachedThreadPool();
```



It can create additional threads when required and reuse idle threads.

Conceptually:

Tasks



Create threads as needed



reuse idle threads



It can grow significantly under load.

Production concern

I generally avoid blindly using:

</> Java

```
Executors.newCachedThreadPool()
```



for high-throughput production APIs because there isn't a strong upper bound on concurrency.

A large number of incoming tasks can result in many threads being created.

Interview answer

"FixedThreadPool is preferable when I need predictable concurrency. CachedThreadPool is useful for many short-lived asynchronous tasks, but because it can create a large number of threads, I would be cautious about using it for high-load production APIs."

8. What is ScheduledExecutorService ?

It is used to execute tasks:

- after a delay
- periodically

Example:

</> Java



```
ScheduledExecutorService scheduler =  
    Executors.newScheduledThreadPool(2);
```

Execute once after delay

</> Java



```
scheduler.schedule(  
    () -> System.out.println("Hello"),  
    5,  
    TimeUnit.SECONDS  
);
```

Runs after 5 seconds.

Execute periodically

</> Java



```
scheduler.scheduleAtFixedRate(  
    () -> cleanup(),  
    0,  
    10,  
    TimeUnit.MINUTES  
);
```

Useful for:

- periodic cleanup
- cache refresh
- polling
- background jobs
- scheduled maintenance

9. What happens when the thread pool queue is full?

This is an important production question.

Suppose:

```
Pool size = 10  
Queue size = 100
```



All 10 workers are busy.

Then:

```
Task 1 → queue  
Task 2 → queue  
...  
Task 100 → queue
```



Now the queue is full.

What happens to the next task depends on the **RejectedExecutionHandler**.

Common policies:

AbortPolicy

Default for `ThreadPoolExecutor`.

Throws:

```
</> Java  
  
RejectedExecutionException
```



CallerRunsPolicy

The submitting thread executes the task.

This can provide a form of **backpressure**.

DiscardPolicy

Silently discards the task.

Usually dangerous if the task is important.

DiscardOldestPolicy

Removes the oldest queued task and tries to submit the new task.

Interview answer

"A full queue doesn't automatically mean the task waits forever. Once the executor cannot accept more work according to its capacity, the configured `RejectedExecutionHandler` determines what happens."

10. What is thread pool starvation?

Thread pool starvation occurs when all worker threads are occupied and no thread is available to execute other tasks.

Example:

Pool = 10 threads

10 tasks

↓

all 10 threads make blocking calls

↓

all threads waiting

↓

another task arrives

↓

cannot execute



A particularly dangerous case:

</> Java



```
executor.submit(() -> {  
    Future<String> future = executor.submit(() -> service());  
    return future.get();  
});
```

If the pool is small and all threads do this, the inner tasks may never get a worker.

That's a **thread starvation/deadlock-like situation**.

Prevention

- Avoid unnecessary blocking.
- Use appropriately sized pools.
- Don't use the same tiny pool for unrelated workloads.
- Separate CPU-bound and blocking I/O workloads where appropriate.
- Prefer asynchronous composition when practical.

11. How do you shut down ExecutorService ?

Use:

</> Java



```
executor.shutdown();
```

Then optionally:

</> Java



```
executor.awaitTermination(  
    30,  
    TimeUnit.SECONDS  
);
```

Example:

</> Java



```
executor.shutdown();  
  
try {  
    if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {  
        executor.shutdownNow();  
    }  
}
```

```
} catch (InterruptedException e) {  
    executor.shutdownNow();  
    Thread.currentThread().interrupt();  
}
```

12. shutdown() vs shutdownNow()

shutdown()

Means:

"Stop accepting new tasks, but finish already submitted tasks."

New tasks ❌
Existing tasks ✅



It's graceful.

shutdownNow()

Attempts to stop execution immediately.

New tasks ❌
Queued tasks → returned
Running tasks → interrupted



Important:

shutdownNow() does **not guarantee** that running tasks immediately stop.

The task must respond properly to interruption.

Interview answer

" shutdown() is my normal graceful shutdown mechanism. shutdownNow() is an aggressive shutdown that interrupts running workers and returns tasks that were still queued."

13. What happens if you don't shut down ExecutorService ?

The executor may continue running.

Its worker threads can keep the application alive and consume resources.

Potential problems:

- application doesn't terminate cleanly
- threads remain alive
- resources remain occupied
- memory/resource leaks
- application redeployment can be affected

In Spring Boot, I'd generally let the container manage the lifecycle of configured executors rather than creating unmanaged executors everywhere.

14. How do you size a thread pool?

There isn't one universal number.

It depends on workload.

The first question I ask is:

Is the workload CPU-bound or I/O-bound?

15. CPU-bound vs I/O-bound

CPU-bound

Examples:

- image processing
- encryption
- complex calculations
- compression

The CPU is the bottleneck.

If you have:

8 CPU cores



you generally don't want thousands of active CPU-intensive threads.

A common starting point is around:

number of CPU cores



or sometimes:

cores + 1



Then benchmark and tune.

I/O-bound

Examples:

API → database
API → another microservice
API → external service



Threads spend significant time waiting.

For example:

Thread 1 → waiting for DB
Thread 2 → waiting for API
Thread 3 → waiting for API
Thread 4 → waiting for DB



Therefore, the optimal pool can be larger than the CPU count.

A commonly discussed starting formula is:

$\text{Threads} \approx \text{CPU cores} \times (1 + \text{Wait Time} / \text{Compute Time})$



But this is a **starting point**, not a magic production formula.

You should validate using:

- latency
 - CPU utilization
 - throughput
 - queue depth
 - downstream capacity
 - memory
 - connection-pool limits
-

16. Production scenario

Your API receives 1,000 requests/sec and every request makes a slow downstream API call.

This is the question I'd expect from a strong backend interview.

Don't answer:

"I'll create 1,000 threads."

That's a bad design.

Step 1: Identify the bottleneck

Suppose:

Incoming traffic = 1000 RPS



Downstream latency = 500 ms

A rough concurrency requirement is:

Concurrency $\approx$ RPS $\times$ latency

$= 1000 \times 0.5$

$= 500$ concurrent requests



This is essentially the intuition behind **Little's Law**.

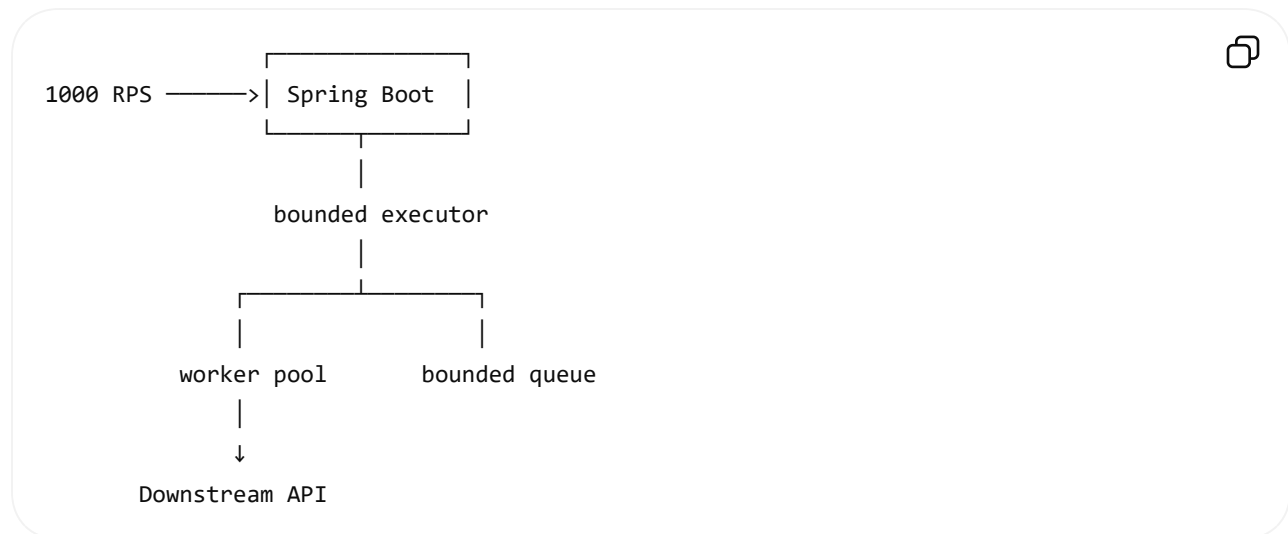
But that does **not** mean:

"Create 500 threads."

That's the important distinction.

17. What would I actually design?

If the downstream call is blocking:



I would use a **bounded executor**, not an unbounded/cached pool.

For example:

```
</> Java
```

```
ThreadPoolExecutor executor =  
    new ThreadPoolExecutor(  
        100,  
        200,  
        60,  
        TimeUnit.SECONDS,  
        new ArrayBlockingQueue<>(500),  
        new ThreadPoolExecutor.CallerRunsPolicy()  
    );
```

The code block shows the configuration of a `ThreadPoolExecutor` in Java. It includes a copy icon in the top right corner.

Those numbers are **illustrative**, not something I'd deploy blindly.

I'd determine them from load testing and downstream limits.

18. Why bounded?

Suppose downstream becomes slow:

```
1000 RPS
↓
downstream slows
↓
requests accumulate
↓
unbounded queue
↓
memory increases
↓
latency increases
↓
OOM / cascading failure
```



A bounded queue gives us controlled backpressure.

19. Very important: protect the downstream service

Suppose your service can technically send:

```
500 concurrent requests
```



but downstream can handle only:

```
100 concurrent requests
```



Then sending 500 is a bad idea.

I'd use mechanisms such as:

```
Bounded concurrency
+
Timeout
+
Circuit breaker
+
Rate limiting
+
Retries with backoff
```



For example:

```
1000 RPS
  ↓
API
  ↓
Concurrency limit
  ↓
100 calls max
  ↓
Downstream
```



This prevents our service from overwhelming the dependency.

20. What if I'm using `CompletableFuture` ?

For blocking downstream calls, don't casually do:

```
</> Java
```

```
CompletableFuture.supplyAsync(() -> slowApi());
```



because without specifying an executor, it uses the common pool.

Instead:

```
</> Java
```

```
CompletableFuture.supplyAsync(
    () -> slowApi(),
    ioExecutor
);
```



where:

```
</> Java
```

```
ExecutorService ioExecutor = ...;
```



This keeps blocking I/O work isolated from unrelated asynchronous work.

21. Even better: use a non-blocking HTTP client when appropriate

If the downstream client supports asynchronous/non-blocking I/O, you may not need one blocking thread per request.

Conceptually:

1000 requests



Non-blocking HTTP client



event-driven I/O



downstream



This can handle many concurrent I/O operations with relatively few threads.

In a modern Java/Spring application, I'd evaluate:

- non-blocking HTTP client
- appropriate connection pool
- timeouts
- bounded concurrency
- resilience mechanisms

rather than solving everything by increasing thread count.

22. Strong 5-year interview answer

If the interviewer asks:

"Your API receives 1,000 requests/sec and each request makes a slow downstream API call. How would you design your thread pool?"

A strong answer would be:

"First I'd identify whether the downstream call is blocking and measure its latency. I wouldn't create one thread per request or use an unbounded cached pool. If the client is blocking, I'd use a bounded I/O-oriented executor with a bounded queue and an appropriate rejection policy. I'd size it based on measured latency, throughput, CPU, memory and, most importantly, the downstream service's concurrency capacity. I'd also

configure connection-pool limits and strict timeouts, and use circuit breaking, rate limiting and controlled retries to prevent cascading failures. If the HTTP client supports non-blocking I/O, I'd consider that instead of simply increasing the thread count. Finally, I'd validate the configuration with load testing and monitor active threads, queue depth, rejection rate, latency and downstream errors."

That's much stronger than simply saying:

"I'll use a thread pool of 100 threads."

Quick interview cheat sheet

Question	Strong short answer
ExecutorService vs manual threads	Reuse threads, control concurrency, manage tasks/lifecycle
execute()	Fire-and-forget, returns void
submit()	Returns Future , captures result/exception
Runnable	No return value
Callable	Returns value, supports checked exceptions
Future	Basic async result, get() can block
CompletableFuture	Async composition, chaining, combining, callbacks
Thread pool	Group of reusable worker threads
FixedThreadPool	Controlled, fixed concurrency
CachedThreadPool	Dynamically creates threads; risky under high load
ScheduledExecutorService	Delayed/periodic tasks
Queue full	Rejection handler decides what happens
Starvation	Workers occupied, other tasks can't progress

Question	Strong short answer
<code>shutdown()</code>	Graceful shutdown
<code>shutdownNow()</code>	Attempts immediate shutdown/interruption
No shutdown	Threads/resources may remain alive
CPU-bound	Usually around CPU-core scale; benchmark
I/O-bound	Can need more concurrency; measure wait/compute and downstream capacity
1000 RPS + slow API	Bounded I/O concurrency + timeouts + backpressure + downstream protection



What is `CompletableFuture`?

Future vs `CompletableFuture`.

`thenApply()` vs `thenAccept()` vs `thenRun()`.

`thenApply()` vs `thenCompose()`.

`thenCombine()` vs `thenCompose()`.

`allOf()` vs `anyOf()`.

`exceptionally()` vs `handle()`.

`whenComplete()`.

How do you handle exceptions in `CompletableFuture`?

Which thread executes `CompletableFuture` tasks?

Common pool vs custom `Executor`.

Show more ▾

For a **5-year Java backend interview**, `CompletableFuture` is very important because interviewers often test whether you understand **async execution, composition, parallel API calls, exception handling, and thread pools**.

1. What is CompletableFuture ?

`CompletableFuture` is a Java class used to perform **asynchronous, non-blocking computations** and then compose their results.

It implements:

</> Java

`Future`
`CompletionStage`



Simple example:

</> Java

```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> "Java");

future.thenAccept(result ->
    System.out.println(result)
);
```



Conceptually:

Start async task



Task running



Result available



continue with next action



The major advantage over `Future` is that you can **chain and combine asynchronous operations**.

2. Future VS CompletableFuture

Future

CompletableFuture

Basic async result

Advanced async pipeline

Future	CompletableFuture
get() commonly used	Can use callbacks/chaining
Composition is difficult	Easy composition
Combining tasks is difficult	thenCombine , allOf , etc.
Limited exception handling	exceptionally , handle , whenComplete
Can cancel	Can cancel
Java 5	Java 8

Future

</> Java



```
Future<String> future =
    executor.submit(() -> callApi());

String result = future.get(); // blocking
```

CompletableFuture

</> Java



```
CompletableFuture
    .supplyAsync(() -> callApi())
    .thenApply(this::transform)
    .thenAccept(System.out::println);
```

Interview answer

" Future represents the result of an asynchronous computation, but composition and exception handling are limited. CompletableFuture provides a richer API for chaining, combining, handling errors and coordinating multiple asynchronous operations."

3. thenApply() vs thenAccept() vs thenRun()

This is a very common question.

thenApply()

Use when you want to **transform the result**.

</> Java



```
CompletableFuture<String> future =  
    CompletableFuture.supplyAsync(() -> "java");  
  
CompletableFuture<Integer> result =  
    future.thenApply(String::length);
```

Flow:

String
↓
transform
↓
Integer



It returns another `CompletableFuture` .

thenAccept()

Use when you **consume the result**, but don't return another result.

</> Java



```
future.thenAccept(result -> {  
    System.out.println(result);  
});
```

Input:

String



Output:

void



thenRun()

Use when you don't care about the previous result.

</> Java



```
future.thenRun(() -> {  
    System.out.println("Completed");  
});
```

It receives **no argument**.

Easy way to remember

thenApply → transform result
thenAccept → consume result
thenRun → just run something



4. thenApply() vs thenCompose()

This is one of the **most important CompletableFuture** questions.

Suppose:

</> Java



```
CompletableFuture<User> getUser()
```

and after getting the user:

</> Java



```
CompletableFuture<Order> getOrder(User user)
```

You want:

User

↓

async call

↓

Order



Use `thenCompose()` .

`</>` **Java**



```
CompletableFuture<Order> result =
    getUser()
        .thenCompose(user -> getOrder(user));
```

Why not `thenApply()` ?

Because:

`</>` **Java**



```
thenApply(user -> getOrder(user))
```

produces:

```
CompletableFuture<CompletableFuture<Order>>
```



That's a nested future.

`thenCompose()` flattens it:

```
CompletableFuture<CompletableFuture<Order>>
    ↓
    thenCompose()
    ↓
CompletableFuture<Order>
```



Rule

`thenApply()` = **synchronous transformation**

`thenCompose()` = **dependent asynchronous operation**

5. `thenCombine()` vs `thenCompose()`

Very important distinction.

thenCompose()

Used when the **second operation depends on the first**.

</> Java



```
getCustomer()
    .thenCompose(customer ->
        getOrders(customer.getId()));
```

Flow:

Customer



Orders



You cannot call the second operation until you have the first result.

thenCombine()

Used when two operations are **independent**.

</> Java

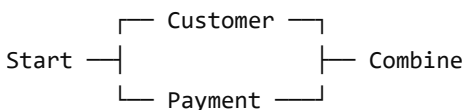


```
CompletableFuture<Customer> customer =
    getCustomer();

CompletableFuture<Payment> payment =
    getPayment();

CompletableFuture<Result> result =
    customer.thenCombine(
        payment,
        (c, p) -> new Result(c, p)
    );
```

Flow:



Remember

Dependent → thenCompose()



Independent → thenCombine()

6. allOf() vs anyOf()

allOf()

Wait until **all futures complete**.

</> Java



```
CompletableFuture.allOf(  
    future1,  
    future2,  
    future3  
);
```

Conceptually:

```
API 1 ———┐  
API 2 ———┤ → all completed  
API 3 ———┘
```



Important:

allOf() returns:

</> Java



```
CompletableFuture<Void>
```

It doesn't directly give you a list of results.

You normally retrieve the individual results afterward.

anyOf()

Completes when **any one future completes**.

</> Java



```
CompletableFuture.anyOf(
    future1,
    future2,
    future3
);
```

Conceptually:

```

API 1  _____
API 2  _____|→ first completed
API 3  _____
  
```



Useful for:

- fastest replica
- multiple equivalent services
- racing alternative sources

Remember

```
allOf → wait for everyone
anyOf → wait for first
```



7. exceptionally() vs handle()

exceptionally()

Used mainly to provide a **fallback when something fails**.

</> Java



```
future
    .exceptionally(ex -> {
        return "DEFAULT";
    });
```

If the previous stage fails:

```
Exception
↓
```



```
exceptionally()
```

↓

```
fallback value
```

handle()

handle() receives **both result and exception**.

```
</> Java

future.handle((result, exception) -> {

    if (exception != null) {
        return "DEFAULT";
    }

    return result;
});
```

So:

```
handle(result, exception)
```

can handle both:

- success
- failure

Difference

```
exceptionally → handle failure
```

```
handle → handle success + failure
```

8. whenComplete()

whenComplete() is generally used when you want to **observe the outcome**, rather than transform it.

</> Java



```
future.whenComplete((result, exception) -> {  
  
    if (exception != null) {  
        log.error("API failed", exception);  
    } else {  
        log.info("API succeeded");  
    }  
});
```

The important distinction is:

```
whenComplete → observe  
handle        → transform/recover
```



For example, logging/metrics are good use cases for `whenComplete()` .

9. How do you handle exceptions in `CompletableFuture` ?

There are three commonly discussed approaches.

`exceptionally()`

</> Java



```
future.exceptionally(ex -> {  
    log.error("Failed", ex);  
    return fallback;  
});
```

`handle()`

</> Java



```
future.handle((result, ex) -> {  
  
    if (ex != null) {  
        return fallback;  
    }  
}
```

```
    return result;
});
```

whenComplete()

</> Java



```
future.whenComplete((result, ex) -> {
    if (ex != null) {
        log.error("Failed", ex);
    }
});
```

Interview answer

"I use exceptionally when I need a failure fallback, handle when I need to transform both success and failure into a result, and whenComplete when I only need side effects such as logging, metrics or tracing."

10. Which thread executes CompletableFuture tasks?

This is subtle and interviewers like it.

There are two forms.

Non-async methods

</> Java



```
future.thenApply(...)
future.thenAccept(...)
future.thenRun(...)
```

The continuation can execute in the thread that completes the previous stage.

For example:

```
Worker thread
  ↓
completes future
  ↓
thenApply()
```



So you should **not assume a particular thread**.

Async methods

</> **Java**



```
future.thenApplyAsync(...)
future.thenAcceptAsync(...)
future.thenRunAsync(...)
```

Without an explicit executor, asynchronous stages typically use the **ForkJoinPool.commonPool()**.

Example:

</> **Java**



```
future.thenApplyAsync(result -> process(result));
```

11. Common Pool vs Custom Executor

By default:

</> **Java**

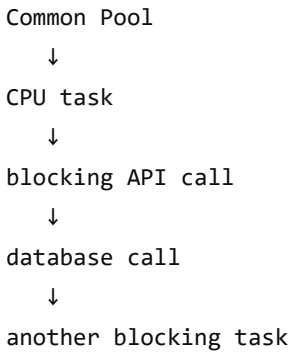


```
CompletableFuture.supplyAsync(() -> callApi());
```

uses the common ForkJoinPool for asynchronous execution.

Problem

Suppose:



```
graph TD; A[Common Pool] --> B[CPU task]; B --> C[blocking API call]; C --> D[database call]; D --> E[another blocking task];
```

Blocking operations can consume common-pool workers and affect unrelated async work.

For production applications, I often prefer:

 **</> Java**

```
ExecutorService ioExecutor =  
    Executors.newFixedThreadPool(50);
```

Then:

 **</> Java**

```
CompletableFuture.supplyAsync(  
    () -> callApi(),  
    ioExecutor  
);
```

This gives explicit control over:

- thread count
- queue
- rejection policy
- lifecycle

Interview answer

"I use the common pool for lightweight asynchronous CPU-oriented work when appropriate. For blocking I/O or production workloads where I need isolation and predictable concurrency, I prefer a custom bounded executor."

12. How would you execute 3 independent APIs in parallel?

Suppose:

Customer API
Order API
Payment API



They are independent.

Don't do:

</> Java



```
Customer customer = customerApi();  
Order order = orderApi();  
Payment payment = paymentApi();
```

because that's sequential:

Customer → Order → Payment



Total $\approx$ T1 + T2 + T3

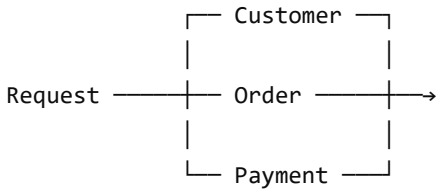
Instead:

</> Java



```
CompletableFuture<Customer> customerFuture =  
    CompletableFuture.supplyAsync(  
        () -> customerApi(),  
        executor  
    );  
  
CompletableFuture<Order> orderFuture =  
    CompletableFuture.supplyAsync(  
        () -> orderApi(),  
        executor  
    );  
  
CompletableFuture<Payment> paymentFuture =  
    CompletableFuture.supplyAsync(  
        () -> paymentApi(),  
        executor  
    );
```

Now:



They execute concurrently.

13. How would you wait for all APIs?

Use:

</> Java

```
CompletableFuture.allOf(  
    customerFuture,  
    orderFuture,  
    paymentFuture  
).join();
```

Then retrieve:

</> Java

```
Customer customer = customerFuture.join();  
Order order = orderFuture.join();  
Payment payment = paymentFuture.join();
```

14. What happens if one API fails?

Suppose:

```
Customer → SUCCESS  
Order    → SUCCESS  
Payment  → FAILURE
```

If you use:

</> Java

```
CompletableFuture.allOf(...)
```

the combined future completes exceptionally when one or more constituent futures fail.

But the other futures aren't automatically cancelled merely because one failed.

So you need to decide your business behavior.

Option 1: Fail the entire request

For example:

Payment is mandatory, so return an error if payment API fails.

Option 2: Fallback

For example:

```
</> Java

paymentFuture.exceptionally(ex ->
    Payment.unavailable()
);
```



Then you can still return the customer/order information.

This decision is **business-specific**.

15. How would you implement timeout?

Modern Java provides:

```
</> Java

orTimeout()
```



Example:

```
</> Java

CompletableFuture<String> future =
    CompletableFuture.supplyAsync(
        () -> slowApi(),
        executor
```



```
);  
  
future.orTimeout(  
    2,  
    TimeUnit.SECONDS  
);
```

If it doesn't complete within 2 seconds, the future completes exceptionally with a timeout.

`completeOnTimeout()`

Instead of failing:

```
</> Java  
  
future.completeOnTimeout(  
    "DEFAULT",  
    2,  
    TimeUnit.SECONDS  
);
```

So:

```
orTimeout  
↓  
timeout → exception  
  
completeOnTimeout  
↓  
timeout → fallback value
```

16. Complete interview problem

Requirement

Call:

```
Customer API  
Order API  
Payment API
```

All are independent.

Return only after all three complete.

Production-style implementation

</> Java



```
public CompletableFuture<CustomerOrderResponse> getDetails() {

    CompletableFuture<Customer> customerFuture =
        CompletableFuture.supplyAsync(
            () -> customerApi.getCustomer(),
            ioExecutor
        ).orTimeout(2, TimeUnit.SECONDS);

    CompletableFuture<Order> orderFuture =
        CompletableFuture.supplyAsync(
            () -> orderApi.getOrders(),
            ioExecutor
        ).orTimeout(2, TimeUnit.SECONDS);

    CompletableFuture<Payment> paymentFuture =
        CompletableFuture.supplyAsync(
            () -> paymentApi.getPayments(),
            ioExecutor
        ).orTimeout(2, TimeUnit.SECONDS);

    return CompletableFuture.allOf(
        customerFuture,
        orderFuture,
        paymentFuture
    )
        .thenApply(v -> {

            Customer customer = customerFuture.join();
            Order order = orderFuture.join();
            Payment payment = paymentFuture.join();

            return new CustomerOrderResponse(
                customer,
                order,
                payment
            );
        });
}
```

```
    });  
}
```

The important thing is that the three calls are **started before waiting for any of them**.

17. What is happening internally?

Suppose:

```
Customer = 200 ms  
Order    = 500 ms  
Payment  = 300 ms
```



Sequential approach

```
Customer 200  
  ↓  
Order 500  
  ↓  
Payment 300  
  
Total = 1000 ms
```



Concurrent approach

```
Customer — 200ms ———┐  
Order    — 500ms ———┤ → Response  
Payment  — 300ms ———┘
```



Approximate completion time:

```
max(200, 500, 300)  
≈ 500 ms
```

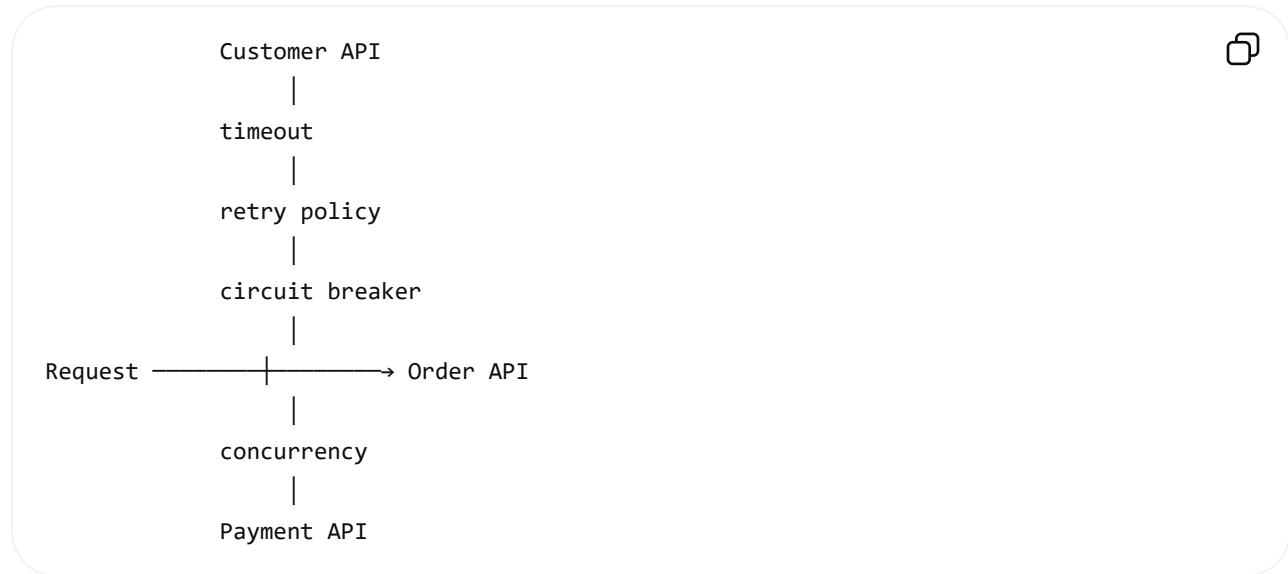


plus application overhead.

That's the major advantage of parallelizing **independent** calls.

18. Stronger production version

For a real microservice, I'd add:



And monitor:

- downstream latency
- timeout rate
- error rate
- executor queue size
- active threads
- rejected tasks
- connection-pool utilization

Also, retries should be carefully controlled because retrying 1,000 failed requests can turn an existing downstream problem into a **retry storm**.

19. Most important CompletableFuture interview cheat sheet

Method	Meaning
<code>supplyAsync()</code>	Start async task that returns result
<code>runAsync()</code>	Start async task with no result
<code>thenApply()</code>	Transform result
<code>thenAccept()</code>	Consume result

Method	Meaning
<code>thenRun()</code>	Run action without result
<code>thenCompose()</code>	Chain dependent async operations
<code>thenCombine()</code>	Combine two independent futures
<code>allOf()</code>	Wait for all
<code>anyOf()</code>	First completed
<code>exceptionally()</code>	Recover from exception
<code>handle()</code>	Handle success + failure
<code>whenComplete()</code>	Observe completion
<code>orTimeout()</code>	Fail after timeout
<code>completeOnTimeout()</code>	Return fallback after timeout

The 5 rules I'd memorize

`thenApply` → transform
`thenCompose` → dependent async call
`thenCombine` → independent async calls
`allOf` → wait for all
`anyOf` → wait for first



And for the interview scenario:

"For three independent downstream APIs, I would start all three `CompletableFutures` immediately using an appropriate custom I/O executor, apply timeouts, use `allOf()` to coordinate completion, then collect the individual results. If one fails, I'd decide based on business requirements whether the entire operation should fail or whether that dependency can have a fallback."



ChatGPT can make mistakes. Check important info.